{EPITECH}

< G-CPP-500 · ADVANCED C++ />

KICK-OFF
R-TYPE PARTIE 2_

G5 · Advanced C++

WE POWER TECH_

{ <TECH TOGETHER TOMORROW> }

< Partie II />

PARTIE II : VUE D'ENSEMBLE_

+ < Prérequis /> Votre prototype fonctionne, réseau compris !

3 TRACKS TECHNIQUES

- Chacune avec plusieurs sujets
- Vous choisissez les sujets sur lesquels travailler
- Vous devrez sans doute refactoriser (et c'est voulu)

LA PLUPART DES SUJETS SONT ORTHOGONAUX

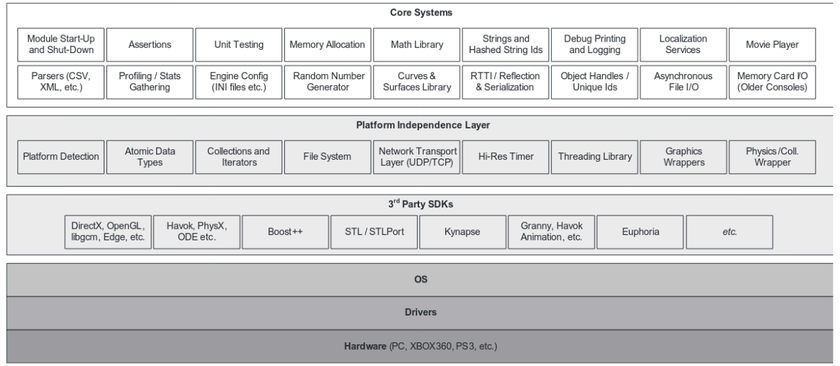
- Il y a de la place pour tout le groupe
- Place au travail en parallèle !

ARCHITECTURE AVANCÉE_

ANATOMIE D'UN MOTEUR_

UN FRAMEWORK POUR CRÉER DES JEUX

- **Abstractions matérielles**

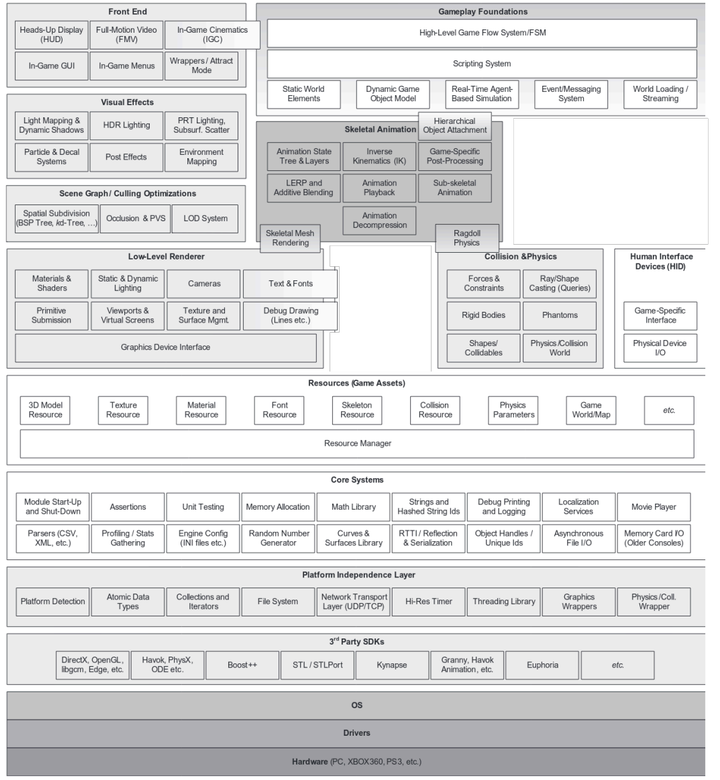


Jason Gregory, Game Engine Architecture

ANATOMIE D'UN MOTEUR_

UN FRAMEWORK POUR CRÉER DES JEUX

- Abstractions matérielles
- **Des « briques » de bas niveau (renderer, physique, son, etc.)**

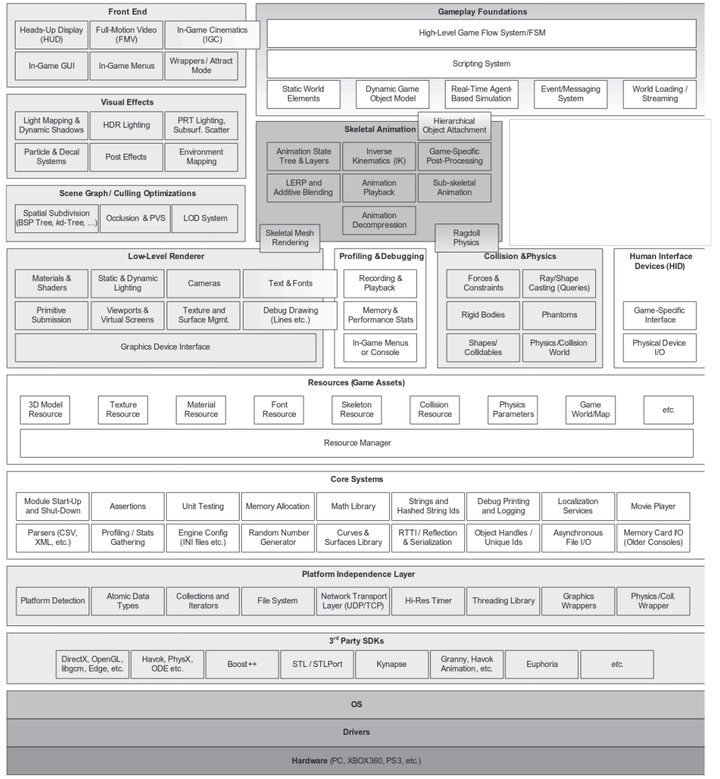


Jason Gregory, Game Engine Architecture

ANATOMIE D'UN MOTEUR_

UN FRAMEWORK POUR CRÉER DES JEUX

- Abstractions matérielles
- Des « briques » de bas niveau (renderer, physique, son, etc.)
- **Outils de développement (debugger, profiler, etc.)**

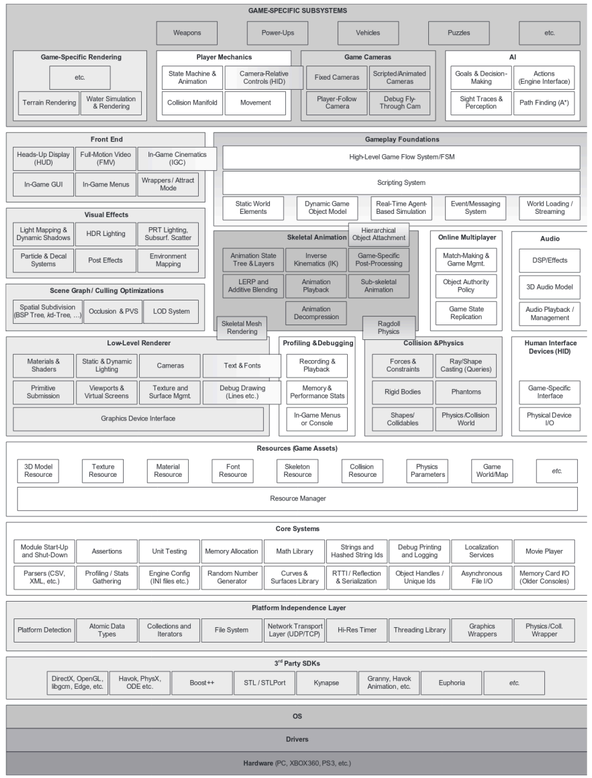


Jason Gregory, Game Engine Architecture

ANATOMIE D'UN MOTEUR_

UN FRAMEWORK POUR CRÉER DES JEUX

- Abstractions matérielles
- Des « briques » de bas niveau (renderer, physique, son, etc.)
- Outils de développement (debugger, profiler, etc.)
- **Suite d'outils de création de contenu (éditeur de maps et d'assets)**
- **Et bien plus encore (IA, cinématiques, plugin API, etc.)**



Jason Gregory, Game Engine Architecture

COMPOSANTS CORE / COMMUNS_

BRIQUES DE BASE

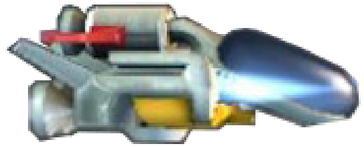
- **Asset manager** améliorer la gestion mémoire



1: baby_force.png



2: force.png

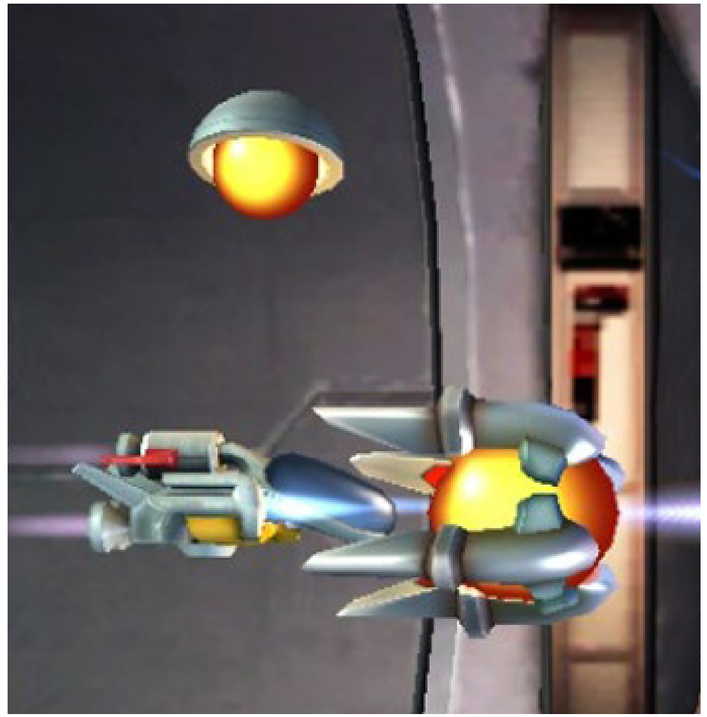


3: player.png

COMPOSANTS CORE / COMMUNS_

BRIQUES DE BASE

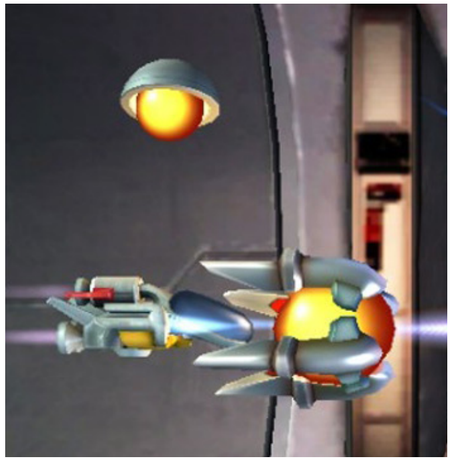
- **Asset manager** améliorer la gestion mémoire
- **Renderer** gérer graphismes, shaders, animation, etc.



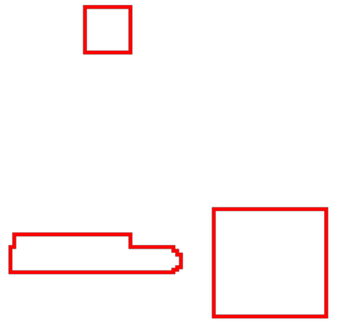
COMPOSANTS CORE / COMMUNS_

BRIQUES DE BASE

- **Asset manager** améliorer la gestion mémoire
- **Renderer** gérer graphismes, shaders, animation, etc.
- **Physics engine** calculer les collisions



Ce que le renderer dessine



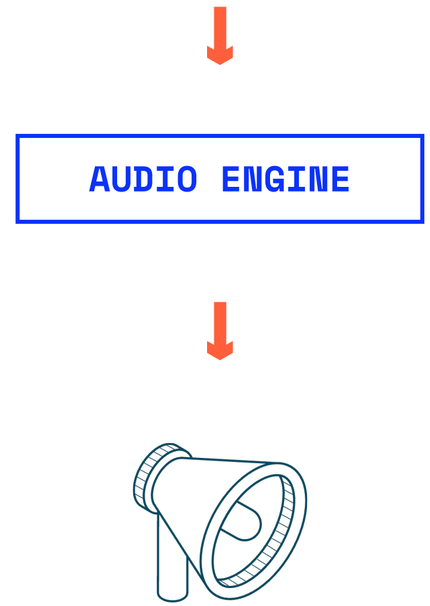
Ce que le physics engine voit

COMPOSANTS CORE / COMMUNS_

BRIQUES DE BASE

- **Asset manager** améliorer la gestion mémoire
- **Renderer** gérer graphismes, shaders, animation, etc.
- **Physics engine** calculer les collisions
- **Audio engine** de la musique à l'audio spatialisé

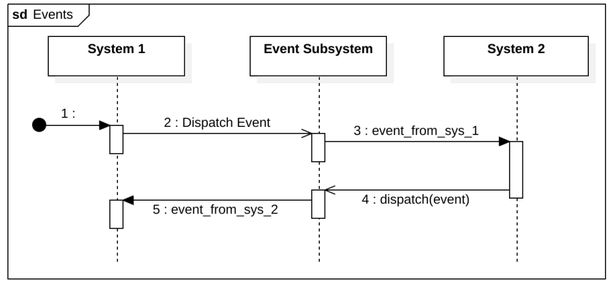
```
play("BGM.wav")
start_fx("engine.wav")
start_fx(player_sound_3)
```



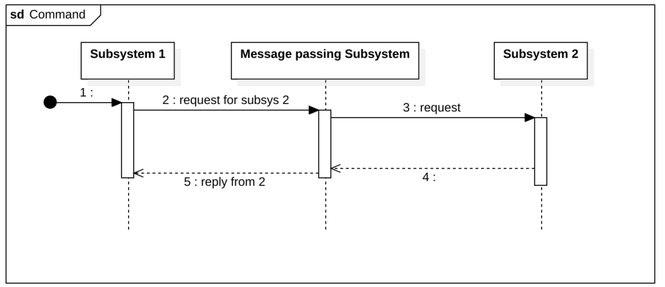
COMPOSANTS CORE / COMMUNS_

BRIQUES DE BASE

- **Asset manager** améliorer la gestion mémoire
- **Renderer** gérer graphismes, shaders, animation, etc.
- **Physics engine** calculer les collisions
- **Audio engine** de la musique à l'audio spatialisé
- **Message subsystem** communication interne
- **Scripting system / plugin API** modularité et extensibilité



Events : il s'est passé quelque chose (peut être async)



Commands : quelque chose doit être fait (rarement async)

OBJECTIFS DE LA TRACK_

PROJET AUTONOME : AUCUNE DÉPENDANCE À TÉLÉCHARGER À LA MAIN.

1. EXTRAIRE LA LOGIQUE DE JEU DU MOTEUR

Il est peut-être temps de les séparer en plusieurs dépôts ou projets.

2. RENDRE LE MOTEUR MODULAIRE

Le serveur n'a besoin d'aucun code de rendu ;)

3. AJOUTER DES MODULES ET DES OUTILS

Lâchez-vous et amusez-vous sur cette partie.

4. MONTRER VOS NOUVELLES FEATURES DANS UN SECOND « JEU »

Pas besoin de coder Dofus, un simple tower defense peut suffire ;)



RÉSEAU AVANCÉ_

RÉSEAU AVANCÉ_

3 SUJETS POSSIBLES À EXPLORER : CHOISISSEZ VOTRE POISON !

SUJET #1

SERVEUR MULTI-INSTANCE

SUJET #2

OPTIMISATIONS RÉSEAU

SUJET #3

LAG COMPENSATION

POUR EN SAVOIR PLUS
Lisez l'annexe.

CONDITIONS DE TEST
Utilisez Clumsy ou netem (voir plus loin).

SUJET #1_

SUJET #1

SERVEUR MULTI-INSTANCE

SERVEUR MULTI-INSTANCE

- Système de lobby / rooms et matchmaking
- Serveur multithreadé (plusieurs instances de jeu en parallèle)
- Fonctions d'administration du serveur

SUJET #2_

CONSOMMATION DE BANDE PASSANTE

- Encodage efficace des données (bit packing, quantization, etc.)
- Compression des données (LZ4, zlib, RLE, etc.)
- Quelle est la taille moyenne de vos paquets UDP ?

FIABILITÉ

- Gérer les pertes, le réordonnancement et la duplication de paquets
- Savez-vous envoyer des messages fiables… sans TCP ?

CONDITIONS DE TEST

Utilisez Clumsy ou netem, avec une bande passante très faible (1 Mbit/s).

SUJET #3_

LAG COMPENSATION : DE NOMBREUX CONCEPTS ET TECHNIQUES

- Client-side prediction
- Interpolation / extrapolation de l'état des entités
- Server reconciliation, synchronisation d'horloge
- Input delaying
- Rollback / replay

ENCORE UNE FOIS

CHOISISSEZ VOTRE POISON !

CONDITIONS DE TEST

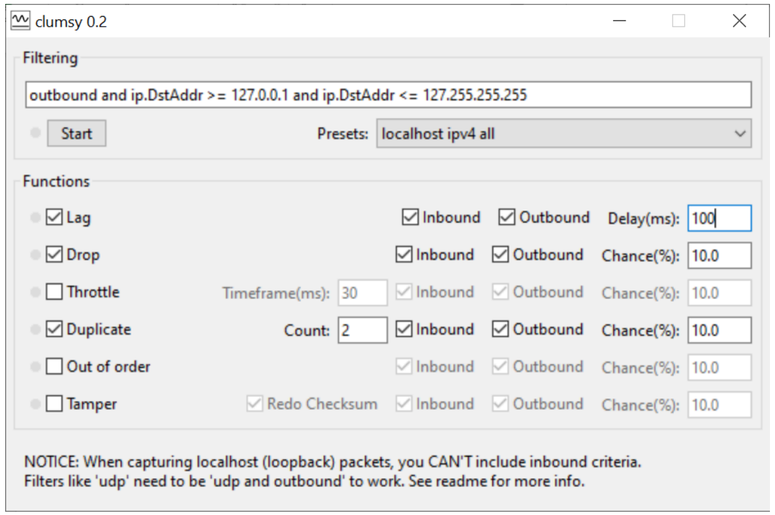
Utilisez Clumsy ou netem, avec une forte latence (> 100 ms).

OUTILS DE SIMULATION RÉSEAU_

SIMULER BANDE PASSANTE FAIBLE, LATENCE, PERTES, DUPLICATION, ALTÉRATION DE PAQUETS, ETC.

OUTILS DISPONIBLES

- **Windows** Clumsy
- **Linux** netem



+ < On testera /> Testez bien votre projet avec ces outils... parce que nous, on le fera !

GAME DESIGN AVANCÉ_

AJOUTER DU CONTENU_

ENNEMIS

- Nouveaux ennemis
- Évolution selon le niveau
- Boss de fin de niveau

NIVEAUX

- Contenu fait main
- Génération procédurale
- Mode infini

ARMES

- Plus de bonus
- Plus d'armes
- Interaction avec les ennemis / boss

RÉGLAGES

- Difficulté
- Tir allié (friendly fire)
- Remappage des touches

AJOUTER DU CONTENU_

DONNÉES

UTILISER LA CONFIGURATION COMME CONTENU

Le JSON s'y prête bien

SCRIPTING

DU SCRIPTING PLUTÔT QUE DU CODE COMPILÉ

MODS

PERMETTRE PLUGINS ET MODS, PAR SCRIPTING ET PAR CODE

QUESTIONS ?_

G5 · Advanced C++

{EPITECH}